

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Аналитическая часть	6
1.1. Постановка задачи	6
1.2. Планирование в режиме реального времени	6
1.3. Анализ структур ядра, предоставляющих информацию о приоритетах, времени выполнения и простоя процессов	7
1.3.1. Структура task_struct	7
1.3.2. Структура sched_info	12
1.3.3. Структура sched_statistics	12
1.4. Анализ функций ядра, позволяющих определить принадлежность задачи к группе задач реального времени	13
1.4.1. Функция rt_prio	13
1.4.2. Функция rt_task	14
1.4.3. Функция task_is_realtime	14
1.5. Передача данных из пространства ядра в пространство пользователя	15
2 Конструкторская часть	18
2.1. Диаграммы состояний	18
2.2. Алгоритмы определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов	19
2.3. Алгоритм предоставления информации о процессах пользователю	21
2.4. Структура программного обеспечения	22
3 Технологическая часть	24
3.1. Выбор языка и среды программирования	24
3.2. Реализация алгоритмов определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов	24

3.3. Реализация алгоритма предоставления информации о процессах пользователю	26
3.4. Makefile	27
3.5. Демонстрация работы	28
4 Исследовательская часть	30
4.1. Описание проводимого исследования	30
4.2. Вывод информации о процессе, выполняющем воспроизведение видео с использованием VLC Media Player	30
4.3. Вывод информации о процессе, выполняющем обработку сетевых запросов	32
4.4. Вывод информации о процессе, выполняющем математические вычисления	34
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39
ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ	40

ВВЕДЕНИЕ

Когда компьютер работает в многозадачном режиме, на нем зачастую запускается сразу несколько процессов или потоков, претендующих на использование центрального процессора. Такая ситуация складывается в том случае, если в состоянии готовности одновременно находятся два или более процесса или потока. Если доступен только один центральный процессор, необходимо выбрать, какой из этих процессов будет выполняться следующим. Та часть операционной системы, на которую возложен этот выбор, называется **планировщиком**, а алгоритм, который он использует, называется **алгоритмом планирования**. В решении задачи планирования процессов ключевую роль играют приоритеты, время непрерывной работы, простоя и блокировки процессов. [1]

Linux является операционной системой разделения времени. Псевдопараллельное выполнение процессов обеспечивается за счёт квантования процессорного времени, позволяющего переключаться с одного процесса на другой за очень короткий промежуток времени.

Целью данной работы является разработка загружаемого модуля ядра Linux для мониторинга изменения приоритетов, времени непрерывной работы, простоя и блокировки процессов.

1 Аналитическая часть

1.1. Постановка задачи

В соответствии с заданием на курсовую работу необходимо разработать загружаемый модуль ядра ОС Linux для мониторинга изменения приоритетов, времени непрерывной работы, простоя и блокировки процессов. Для решения данной задачи необходимо

- проанализировать и выбрать структуры ядра, содержащие необходимую информацию;
- проанализировать и выбрать методы передачи информации из модуля ядра в пространство пользователя;
- разработать алгоритмы и структуру программного обеспечения;
- исследовать приложения различного характера с помощью разработанного программного обеспечения.

1.2. Планирование в режиме реального времени

Проблема планирования в режиме реального времени заключается в том, что группы процессов должны полагаться на постоянство пропускной способности (например, времени исполнения). Для планирования нескольких групп задач в реальном времени может потребоваться назначение так называемого гарантированного доступного времени исполнения. [2]

В качестве решения поставленной проблемы доступное процессорное время делится посредством указания того, сколько времени на выполнение даётся на указанный период. Данное время выделяется для каждой группы процессов реального времени таким образом, что каждая группа может выполняться только в своё выделенное время. Время, не выделенное группам реального времени или не использованное ими, будет выделено задачам с обычным приоритетом (SCHED_OTHER). [2]

Рассмотрим пример: рендерер реального времени с фиксированным количеством кадров должен выдавать 25 кадров в секунду, что даёт период 0.04 секунды на каждый кадр. Если рендерер также должен проигрывать аудио и

реагировать на пользовательский ввод, оставляя около 80% доступного процессорного времени на работу с графикой, то для этой группы можно выделить время выполнения $0.8 \cdot 0.04 = 0.032$ секунды. [2]

Таким образом, «графической группе» будет выделен период 0.04 секунды с ограничением времени выполнения 0.032 секунды. Если аудиопотоку необходимо заполнять буфер DMA (Direct Memory Access, прямой доступ к памяти) каждые 0.005 секунд, но для этого требуется около 3% времени процессора, ему может быть выделено время выполнения $0.03 \cdot 0.005 = 0.00015$ секунд. Таким образом, данная группа может быть запланирована с периодом 0.005 секунд с временем выполнения 0.00015 секунд. [2]

Оставшееся процессорное время будет использоваться для обработки пользовательского ввода и других задач. Поскольку задачам реального времени явно выделено процессорное время, необходимое для выполнения их задач, нехватка буфера в графике или аудио может быть исключена. [2]

Однако на текущий момент приведенный выше пример ещё не реализован полностью в силу того, что отсутствует реализация планировщика EDF (Earliest Deadline First scheduling, алгоритм планирования по ближайшему сроку завершения) для использования неоднородных периодов. [2]

1.3. Анализ структур ядра, предоставляющих информацию о приоритетах, времени выполнения и простоя процессов

1.3.1. Структура `task_struct`

В ядре Linux процесс описывается структурой `task_struct`. Она определена в файле `include/linux/sched.h`. Ниже перечислены требующиеся в работе поля данной структуры (версия ядра 6.1.0). [3]

1. **pid** (Process Identifier) — уникальный идентификатор процесса, по которому можно получить информацию об этом процессе, а также направить ему управляющий сигнал или завершить его.
2. **prio** — значение, которое использует планировщик для принятия решений, связанных с планированием рассматриваемого процесса. Чем ниже значение `prio`, тем выше приоритет процесса. Константы, задающие диапазон значений приоритетов, представлены в листинге 1.1. [4]

Листинг 1.1 – Константы, задающие диапазон значений приоритетов процессов, версия ядра 6.1.0

```
1 #define MAX_NICE      19
2 #define MIN_NICE      -20
3 #define NICE_WIDTH    (MAX_NICE - MIN_NICE + 1)
4
5 /*
6  * Priority of a process goes from 0..MAX_PRIO-1, valid RT
7  * priority is 0..MAX_RT_PRIO-1, and SCHED_NORMAL/SCHED_BATCH
8  * tasks are in the range MAX_RT_PRIO..MAX_PRIO-1. Priority
9  * values are inverted: lower p->prio value means higher priority.
10 */
11
12 #define MAX_RT_PRIO    100
13
14 #define MAX_PRIO        (MAX_RT_PRIO + NICE_WIDTH)
15 #define DEFAULT_PRIO    (MAX_RT_PRIO + NICE_WIDTH / 2)
16
17 /*
18  * Convert user-nice values [ -20 ... 0 ... 19 ]
19  * to static priority [ MAX_RT_PRIO..MAX_PRIO-1 ],
20  * and back.
21 */
22 #define NICE_TO_PRIO(nice) ((nice) + DEFAULT_PRIO)
23 #define PRIO_TO_NICE(prio) ((prio) - DEFAULT_PRIO)
```

Значение **nice** (фактор «любезности») — целое число в диапазоне от -20 до 19 (по умолчанию 0). Чем меньше значение фактора «любезности» процесса, тем выше его приоритет. Фоновым процессам задаются более высокие значения фактора «любезности».

Диапазон значений **prio** может быть поделён на два интервала:

- от 0 до 99 – процесс реального времени;
- от 100 до 139 – обычный процесс.

Реализации функций для вычисления приоритета процесса представлены в листинге 1.2. [5]

Листинг 1.2 – Реализации функций для вычисления приоритета процесса, версия ядра 6.1.0

```
1 /* kernel prio, less is more */
2 static inline int __task_prio(struct task_struct *p)
3 {
```

```

4      if (p->sched_class == &stop_sched_class) /* trumps deadline */
5          return -2;
6
7      if (rt_prio(p->prio)) /* includes deadline */
8          return p->prio; /* [-1, 99] */
9
10     if (p->sched_class == &idle_sched_class)
11         return MAX_RT_PRIO + NICE_WIDTH; /* 140 */
12
13     return MAX_RT_PRIO + MAX_NICE; /* 120, squash fair */
14 }
15
16 /*
17  * Calculate the current priority, i.e. the priority
18  * taken into account by the scheduler. This value might
19  * be boosted by RT tasks, or might be boosted by
20  * interactivity modifiers. Will be RT if the task got
21  * RT-boosted. If not then it returns p->normal_prio.
22  */
23 static int effective_prio(struct task_struct *p)
24 {
25     p->normal_prio = normal_prio(p);
26     /*
27      * If we are RT tasks or we were boosted to RT priority,
28      * keep the priority unchanged. Otherwise, update priority
29      * to the normal priority:
30      */
31     if (!rt_prio(p->prio))
32         return p->normal_prio;
33     return p->prio;
34 }

```

3. **static_prio** — значение, которое не изменяется ядром при работе планировщика, однако оно может быть изменено с использованием пользовательского приоритета nice. Реализация функции для изменения значений nice и static_prio процесса представлены в листинге 1.3. [5]

Листинг 1.3 – Реализация функции для изменения значений nice и static_prio процесса, версия ядра 6.1.0

```

1 void set_user_nice(struct task_struct *p, long nice)
2 {
3     bool queued, running;
4     int old_prio;
5     struct rq_flags rf;
6     struct rq *rq;
7

```

```

8      if (task_nice(p) == nice || nice < MIN_NICE || nice > MAX_NICE)
9          return;
10     /*
11     * We have to be careful, if called from sys_setpriority(),
12     * the task might be in the middle of scheduling on another CPU.
13     */
14     rq = task_rq_lock(p, &rf);
15     update_rq_clock(rq);
16
17     /*
18     * The RT priorities are set via sched_setscheduler(), but we still
19     * allow the 'normal' nice value to be set - but as expected
20     * it won't have any effect on scheduling until the task is
21     * SCHED_DEADLINE, SCHED_FIFO or SCHED_RR:
22     */
23     if (task_has_dl_policy(p) || task_has_rt_policy(p)) {
24         p->static_prio = NICE_TO_PRIO(nice);
25         goto out_unlock;
26     }
27     queued = task_on_rq_queued(p);
28     running = task_current(rq, p);
29     if (queued)
30         dequeue_task(rq, p, DEQUEUE_SAVE | DEQUEUE_NOCLOCK);
31     if (running)
32         put_prev_task(rq, p);
33
34     p->static_prio = NICE_TO_PRIO(nice);
35     set_load_weight(p, true);
36     old_prio = p->prio;
37     p->prio = effective_prio(p);
38
39     if (queued)
40         enqueue_task(rq, p, ENQUEUE_RESTORE | ENQUEUE_NOCLOCK);
41     if (running)
42         set_next_task(rq, p);
43
44     /*
45     * If the task increased its priority or is running and
46     * lowered its priority, then reschedule its CPU:
47     */
48     p->sched_class->prio_changed(rq, p, old_prio);
49
50 out_unlock:
51     task_rq_unlock(rq, p, &rf);
52 }

```

4. **normal_prio** — значение, которое зависит от статического приоритета и

политики планировщика задач. Если политикой планирования процесса является `SCHED_DEADLINE`, то его приоритет будет равен -1. Для процессов не реального времени данное значение равняется значению статического приоритета `static_prio`, то есть `120 + nice`. Для процессов реального времени данное значение равняется значению, вычисленному с использованием максимального значения приоритета процесса реального времени и, непосредственно, его `rt_prio`. Важно отметить тот факт, что, чем больше значение `rt_priority` процесса, тем выше его приоритет. Функция вычисления нормального приоритета процесса предоставлена в листинге 1.4. [5]

Листинг 1.4 – Реализация функции для вычисления нормального приоритета процесса, версия ядра 6.1.0

```
1 static inline int __normal_prio(int policy, int rt_prio, int nice)
2 {
3     int prio;
4
5     if (dl_policy(policy))
6         prio = MAX_DL_PRIO - 1;
7     else if (rt_policy(policy))
8         prio = MAX_RT_PRIO - 1 - rt_prio;
9     else
10        prio = NICE_TO_PRIO(nice);
11
12    return prio;
13 }
14
15 /*
16  * Calculate the expected normal priority: i.e. priority
17  * without taking RT-inheritance into account. Might be
18  * boosted by interactivity modifiers. Changes upon fork,
19  * setprio syscalls, and whenever the interactivity
20  * estimator recalculates.
21  */
22 static inline int normal_prio(struct task_struct *p)
23 {
24     return __normal_prio(p->policy, p->rt_priority,
25                          PRIO_TO_NICE(p->static_prio));
26 }
```

5. **utime** — это процессорное время, проведённое в режиме пользователя и затраченное на выполнение команд. Данное значение включает в себя только время, затраченное центральным процессором, и не включает в

себя время, проведённое процессом в очереди на выполнение.

6. **stime** — это процессорное время, проведённое в режиме ядра и затраченное на выполнение системных вызовов при исполнении процесса.

1.3.2. Структура sched_info

В ядре Linux дескриптор процесса (struct task_struct) ссылается на структуру sched_info, предоставляющую информацию о планировании процесса. Определение данной структуры представлено в листинге 1.5. [3]

Листинг 1.5 – Определение struct sched_info, версия ядра 6.1.0

```
1 struct sched_info {
2     #ifdef CONFIG_SCHED_INFO
3     /* Cumulative counters: */
4
5     /* # of times we have run on this CPU: */
6     unsigned long          pcount;
7
8     /* Time spent waiting on a runqueue: */
9     unsigned long long      run_delay;
10
11     /* Timestamps: */
12
13     /* When did we last run on a CPU? */
14     unsigned long long      last_arrival;
15
16     /* When were we last queued to run? */
17     unsigned long long      last_queued;
18
19     #endif /* CONFIG_SCHED_INFO */
20 };
```

В данной работе требуется рассмотреть поле **run_delay**, содержащее количество времени, которое процесс провёл в очереди на выполнение.

1.3.3. Структура sched_statistics

В ядре Linux дескриптор процесса (struct task_struct) ссылается на структуру sched_statistics, предоставляющую информацию о планировании процесса. Она определена в файле include/linux/sched.h. Ниже перечислены требующиеся в работе поля данной структуры (версия ядра 6.1.0). [3]

1. **wait_max** — максимальное время, которое процесс или поток провёл в очереди, ожидая выделения ресурсов.
2. **wait_count** — общее количество раз, когда процесс или поток попадал в очередь на выделение ресурсов.
3. **wait_sum** — суммарное время, которое процесс или поток провёл в очереди на выделение ресурсов.
4. **iowait_count** — количество раз, когда процесс или поток был блокирован в ожидании завершения ввода/вывода.
5. **iowait_sum** — суммарное время, которое процесс или поток был блокирован в ожидании завершения ввода/вывода.
6. **sleep_max** — максимальное время, которое процесс или поток провёл в состоянии прерываемого сна.
7. **sum_sleep_runtime** — суммарное время, которое процесс или поток провёл в состоянии прерываемого сна.
8. **block_max** — максимальное время, которое процесс или поток провёл в состоянии блокировки.
9. **sum_block_runtime** — суммарное время, которое процесс или поток провёл в состоянии блокировки.
10. **exec_max** — максимальное время, которое процесс или поток провёл в состоянии выполнения.

Представленные выше поля структуры `shed_statistics` процесса обновляются во время его выполнения с помощью соответствующих функций, описанных в файле `/kernel/sched/stats.h`. [6]

1.4. Анализ функций ядра, позволяющих определить принадлежность задачи к группе задач реального времени

1.4.1. Функция `rt_prio`

Функция `rt_prio` определяет, является ли процесс задачей реального времени, с использованием значения приоритета. Приоритет процесса сравнивает-

ся с максимальным значением приоритета для задачи реального времени. Если данное условие истинно, то процесс относится к задачам реального времени и возвращается единица, иначе ноль. Стоит отметить, что макрос `unlikely` в данном случае применяется для оптимизации скорости выполнения сравнения. Реализация функции `rt_prio` приведена в листинге 1.6. [7]

Листинг 1.6 – Реализация функции `rt_prio`, версия ядра 6.1.0

```
1      static inline int rt_prio(int prio)
2      {
3          if (unlikely(prio < MAX_RT_PRIO))
4              return 1;
5          return 0;
6      }
```

1.4.2. Функция `rt_task`

Функция `rt_task` обёртывает вызов функции `rt_prio`. Если заданный процесс (`struct task_struct`) является задачей реального времени (приоритет меньше максимального для задачи реального времени), то возвращается единица, иначе ноль. Реализация функции `rt_task` приведена в листинге 1.7. [7]

Листинг 1.7 – Реализация функции `rt_task`, версия ядра 6.1.0

```
1      static inline int rt_task(struct task_struct *p)
2      {
3          return rt_prio(p->prio);
4      }
```

1.4.3. Функция `task_is_realtime`

Функция `task_is_realtime` анализирует политику планирования, установленную для заданного процесса (`struct task_struct`). Планировщик Linux оперирует следующими политиками планирования:

- `SCHED_FIFO` — планирование по алгоритму FIFO (First in-first out);
- `SCHED_RR` — планирование по алгоритму Round-robin;
- `SCHED_DEADLINE` — задание крайнего срока для выполнения спорадических¹ задач;

¹Спорадическая задача — это задача, состоящая из последовательности заданий, каждое из которых активируется не более одного раза за период.

- SCHED_OTHER — планирование разделения времени по умолчанию;
- SCHED_BATCH — планирование пакетных процессов;
- SCHED_IDLE — планирование низкоприоритетных процессов.

SCHED_FIFO и SCHED_RR являются политиками реального времени. [8]

Реализация функции task_is_realtime приведена в листинге 1.8. [7]

Листинг 1.8 – Реализация функции task_is_realtime, версия ядра 6.1.0

```

1      static inline bool task_is_realtime(struct task_struct *tsk)
2      {
3          int policy = tsk->policy;
4
5          if (policy == SCHED_FIFO || policy == SCHED_RR)
6              return true;
7          if (policy == SCHED_DEADLINE)
8              return true;
9          return false;
10     }
```

1.5. Передача данных из пространства ядра в пространство пользователя

В ОС Linux для передачи данных из пространства ядра в пространство пользователя может использоваться виртуальная файловая система proc, которая предоставляет системные вызовы для реализации взаимодействия между двумя этими пространствами.

Структура proc_ops определена в файле linux/proc_fs.h и содержит указатели на функции драйвера, которые отвечают за выполнение различных операций с устройством. Поля структуры представлены в листинге 1.9. [9]

Листинг 1.9 – Определение struct proc_ops, версия ядра 6.1.0

```

1      struct proc_ops {
2          unsigned int proc_flags;
3          int (*proc_open)(struct inode *, struct file *);
4          ssize_t (*proc_read)(struct file *, char __user *, size_t,
5                               loff_t *);
6          ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
7          ssize_t (*proc_write)(struct file *, const char __user *, size_t,
8                                loff_t *);
9          /* mandatory unless nonseekable_open() or equivalent is used */
10         loff_t (*proc_lseek)(struct file *, loff_t, int);
```

```

11     int (*proc_release)(struct inode *, struct file *);
12     __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
13     long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
14     #ifdef CONFIG_COMPAT
15     long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned
16         long);
17     #endif
18     int (*proc_mmap)(struct file *, struct vm_area_struct *);
19     unsigned long (*proc_get_unmapped_area)(struct file *, unsigned
20         long, unsigned long, unsigned long, unsigned long);
21 } __randomize_layout;

```

Функции `copy_to_user` и `copy_from_user`, определённые в файле `include/linux/uaccess.h`, позволяют копировать данные из пространства ядра в пространство пользователя и наоборот. Они возвращают количество байт, которые не удалось скопировать. Реализация данных функций приведена в листинге 1.10. [10]

Листинг 1.10 – Реализация функции `copy_to_user`, версия ядра 6.1.0

```

1     static __always_inline unsigned long __must_check
2     copy_from_user(void *to, const void __user *from, unsigned long n)
3     {
4         if (check_copy_size(to, n, false))
5             n = __copy_from_user(to, from, n);
6         return n;
7     }
8
9     static __always_inline unsigned long __must_check
10    copy_to_user(void __user *to, const void *from, unsigned long n)
11    {
12        if (check_copy_size(from, n, true))
13            n = __copy_to_user(to, from, n);
14        return n;
15    }

```

Вывод из аналитической части

В результате анализа кода ядра определены структуры, содержащие необходимую информацию: `task_struct` (поля `pid`, `prio`, `static_prio`, `normal_prio`, `utime`, `stime`), `sched_info` (поле `run_delay`) и `sched_statistics` (поля `wait_max`, `wait_count`, `wait_sum`, `iowait_count`, `iowait_sum`, `sleep_max`, `sum_sleep_runtime`, `block_max`, `sum_block_runtime`, `exec_max`). Также были определены функции, позволяющие определить, относится ли процесс к задачам реального времени: `rt_prio`,

rt_task и task_is_realtime.

2 Конструкторская часть

2.1. Диаграммы состояний

На рисунках 2.1 и 2.2 представлены диаграммы в нотации IDEF0, отражающие процесс мониторинга приоритетов, времени непрерывной работы, простоя и блокировки процессов.



Рис. 2.1 – Диаграмма IDEF0, нулевой уровень

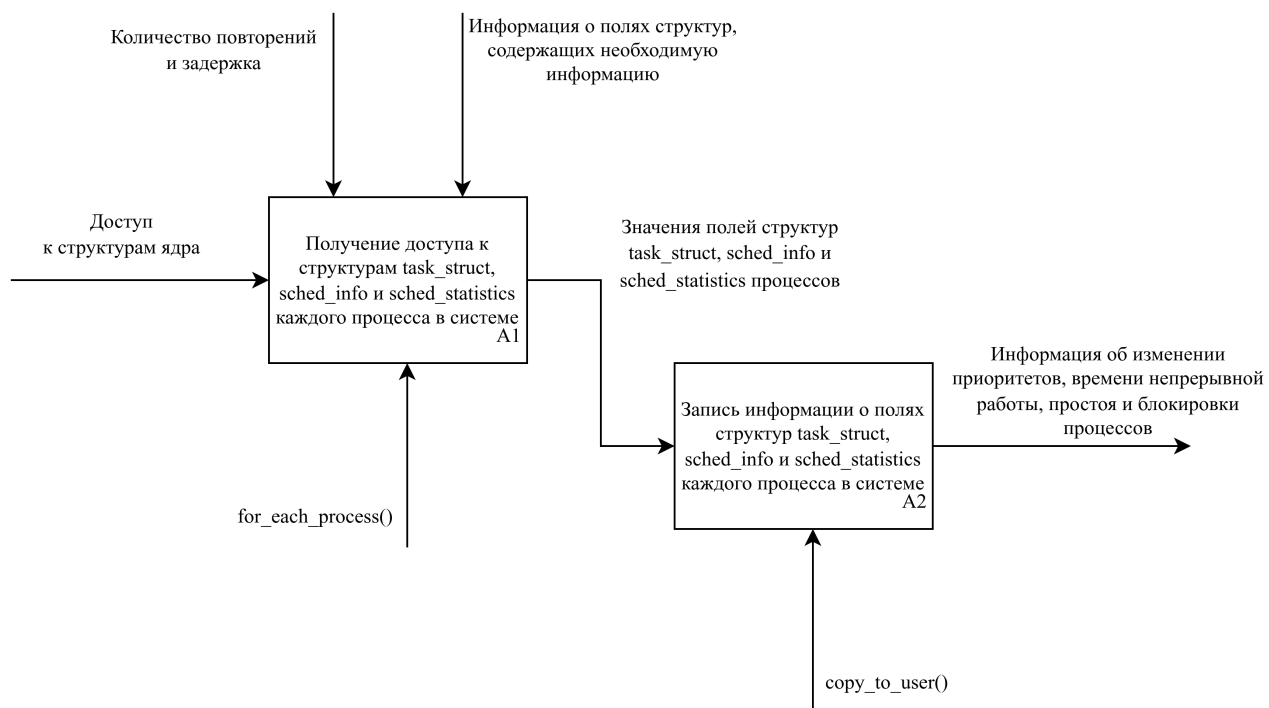


Рис. 2.2 – Диаграмма IDEF0, первый уровень

2.2. Алгоритмы определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов

На рисунках 2.3 и 2.4 представлены алгоритмы определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов.

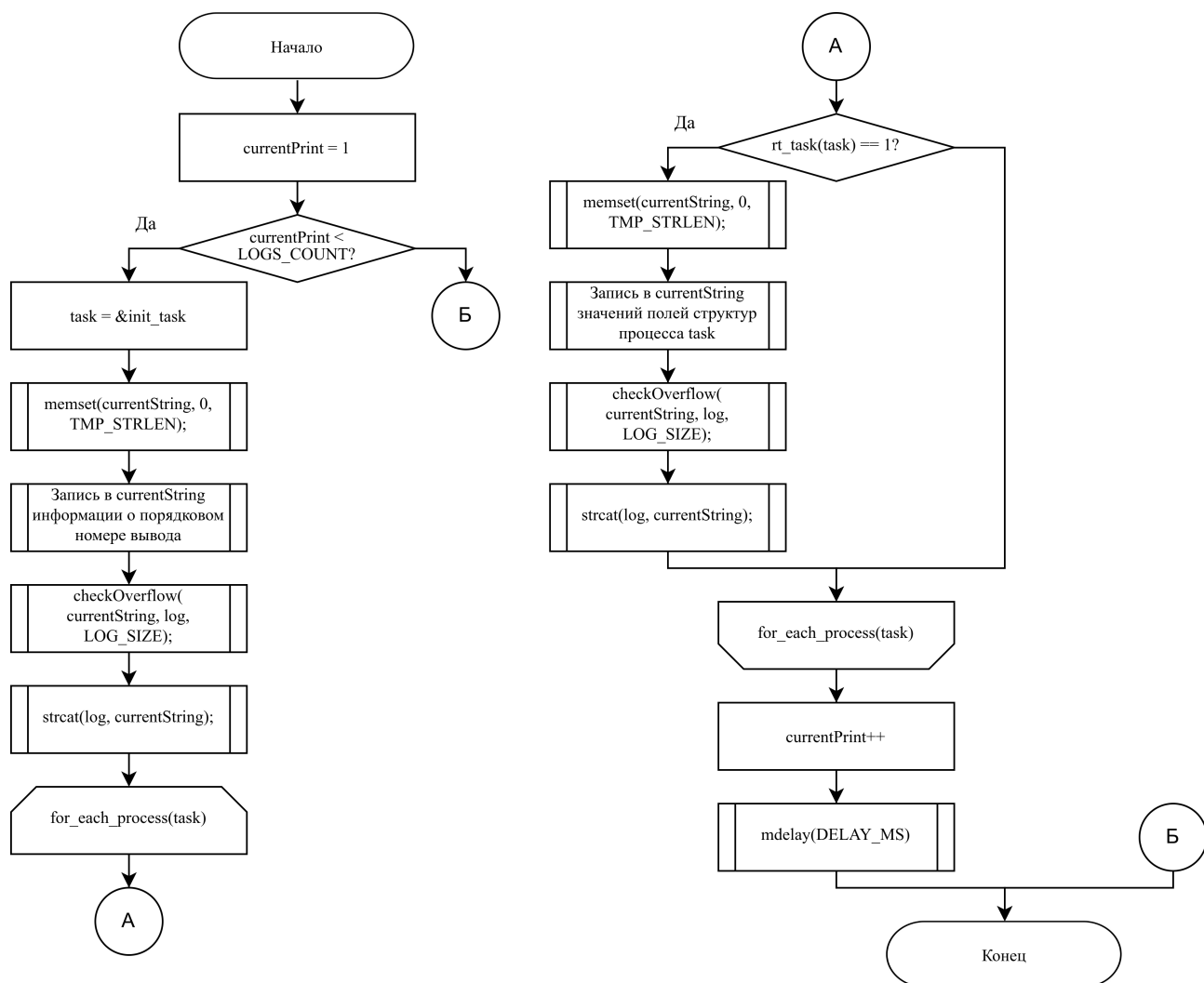


Рис. 2.3 – Алгоритм определения приоритетов, времени непрерывной работы, простоя и блокировки процессов

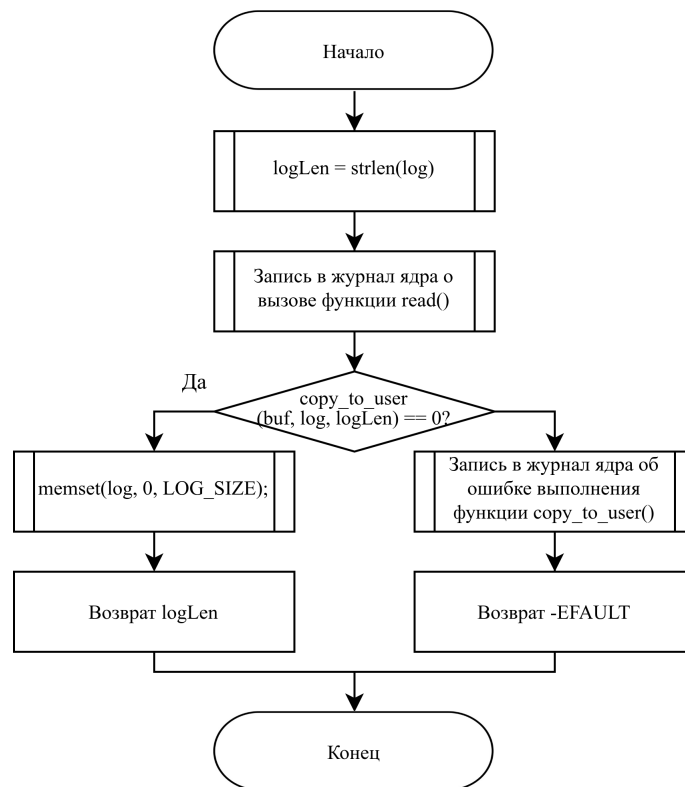


Рис. 2.4 – Алгоритм сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов

2.3. Алгоритм предоставления информации о процессах пользователю

На рисунке 2.5 представлен алгоритм предоставления информации об изменении приоритетов, времени непрерывной работы, простоя и блокировки процессов пользователю.

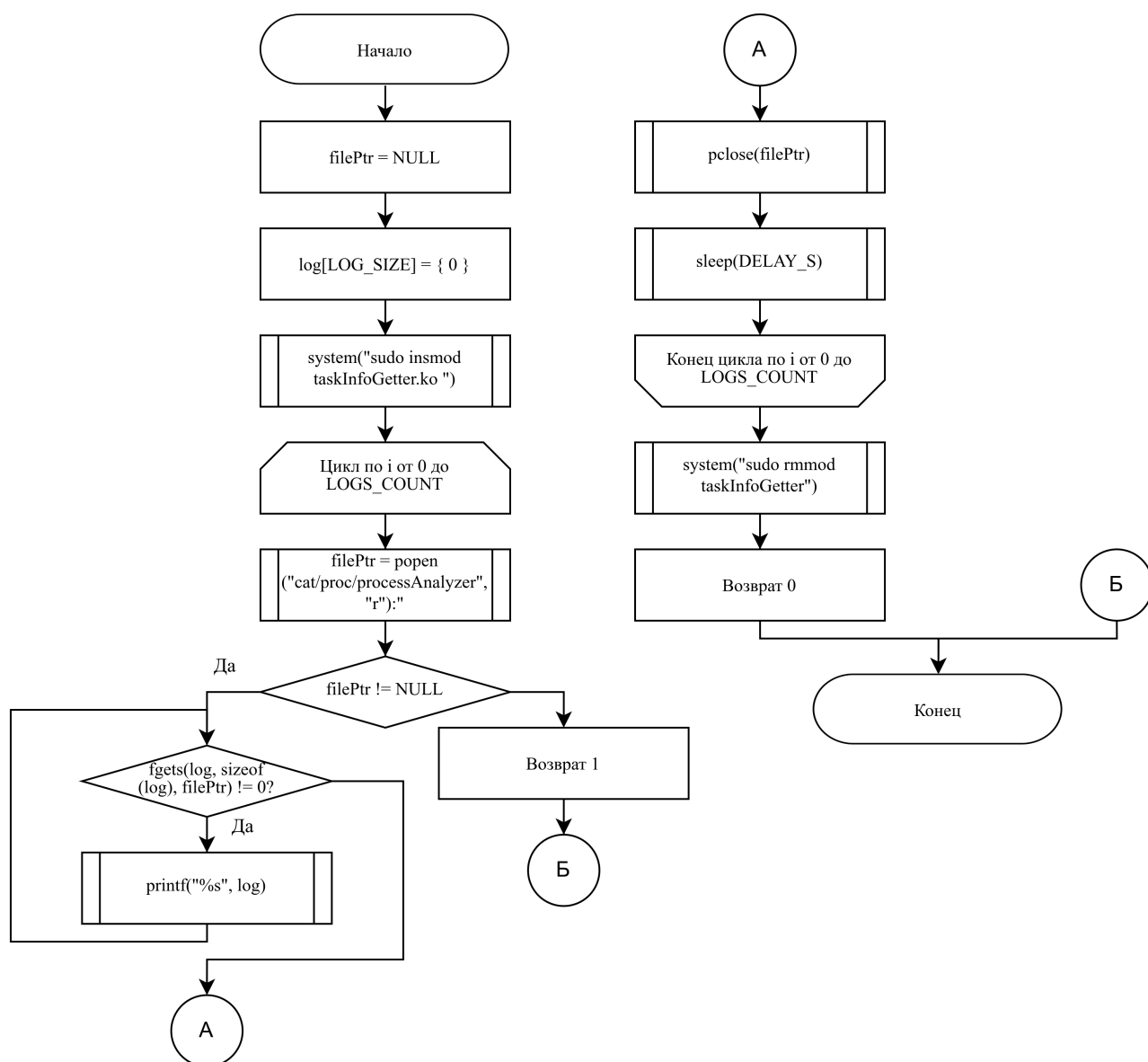


Рис. 2.5 – Алгоритм предоставления информации об изменении приоритетов, времени непрерывной работы, простоя и блокировки процессов пользователю

2.4. Структура программного обеспечения

На рисунке 2.6 представлена структура программного обеспечения.

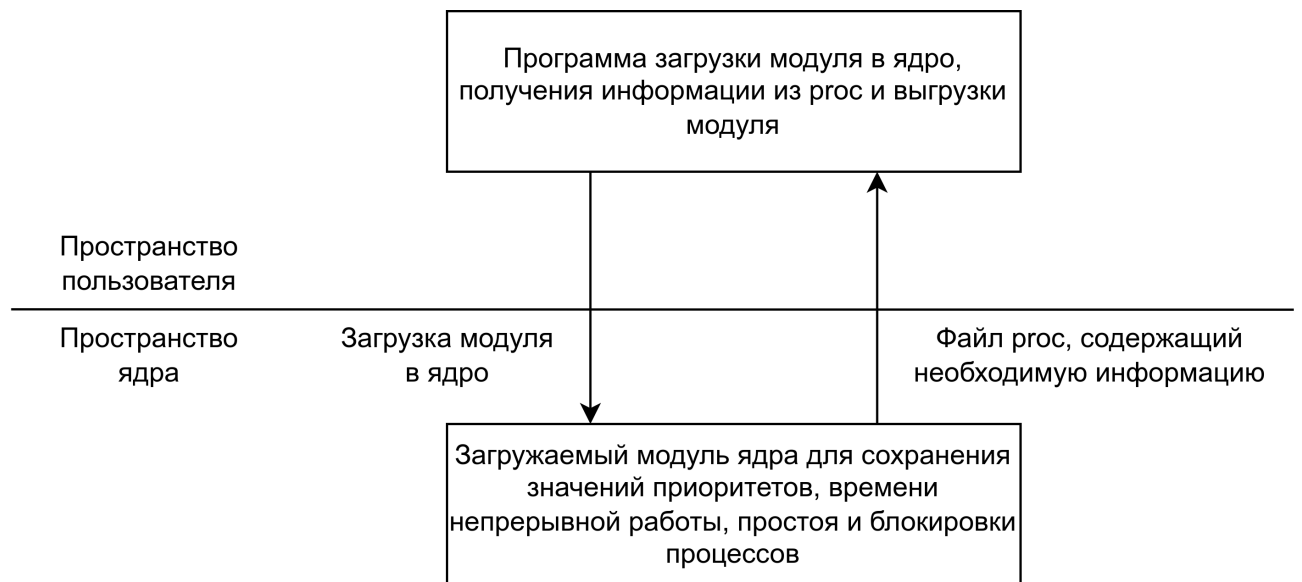


Рис. 2.6 – Структура программного обеспечения

3 Технологическая часть

3.1. Выбор языка и среды программирования

При написании программного кода использовался язык программирования С. [11] Выбор языка С обусловлен тем, что в настоящий момент большая часть исходного кода ядра Linux, его модулей и драйверов написана именно на данном языке. [12]

В качестве компилятора был выбран gcc. [13] Выбор обоснован тем, что данный компилятор является предпочтительным для сборки ядра Linux. [14]

3.2. Реализация алгоритмов определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов

В листинге 3.1 представлена реализация алгоритмов определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов.

Листинг 3.1 – Реализация алгоритмов определения и сохранения приоритетов, времени непрерывной работы, простоя и блокировки процессов

```
1 static int printTasks(void *arg)
2 {
3     struct task_struct *task;
4     size_t currentPrint = 1;
5     char currentString[TMP_STRLEN];
6
7     while (currentPrint <= LOGS_COUNT)
8     {
9         task = &init_task;
10        memset(currentString, 0, TMP_STRLEN);
11        snprintf(currentString, TMP_STRLEN, "~~~~~%lu "
12        "TIME~~~~~\n", currentPrint);
13        checkOverflow(currentString, log, LOG_SIZE);
14        strcat(log, currentString);
15
16        for_each_process(task)
17        {
18            if (task_is_realtime(task))
19            {
20                memset(currentString, 0, TMP_STRLEN);
21                snprintf(currentString, TMP_STRLEN,
22                "pid: %-5d, comm: %15s\nprio: %3d, static_prio: %3d,
```

```

23         normal_prio (with "
24         "scheduler policy): %3d, rt_priority: %3d\n"
25         "run_delay: %10lld\nutime: %10lld (ticks), stime:
26         %15lld (ticks)\n"
27         "sched_statistics:\nwait_max: %10llu, wait_count:
28         %10llu, wait_sum: %10llu\n"
29         "iowait_count: %10llu, iowait_sum: %10llu\n"
30         "sleep_max: %10llu, sum_sleep_runtime: %10llu\n"
31         "block_max: %10llu, sum_block_runtime: %10llu, exec_max:
32         %10llu\n\n",
33         task->pid, task->comm, task->prio, task->static_prio,
34         task->normal_prio, task->rt_priority,
35         task->sched_info.run_delay, task->utime, task->stime,
36         task->stats.wait_max, task->stats.wait_count,
37         task->stats.wait_sum,
38         task->stats.iowait_count, task->stats.iowait_sum,
39         task->stats.sleep_max, task->stats.sum_sleep_runtime,
40         task->stats.block_max, task->stats.sum_block_runtime,
41         task->stats.exec_max
42     );
43     checkOverflow(currentString, log, LOG_SIZE);
44     strcat(log, currentString);
45     }
46 }
47
48     currentPrint++;
49     mdelay(DELAY_MS);
50 }
51
52     return 0;
53 }
54
55 static int analyzerOpen(struct inode *spInode, struct file *spFile)
56 {
57     printk(KERN_INFO "%s open called\n", LOG_PREFIX);
58     try_module_get(THIS_MODULE);
59     return 0;
60 }
61
62 static ssize_t analyzerRead(struct file *filep, char __user *buf,
63     size_t count, loff_t *offp)
64 {
65     ssize_t logLen = strlen(log);
66     printk(KERN_INFO "%s read called\n", LOG_PREFIX);
67
68     if (copy_to_user(buf, log, logLen))
69     {
70         printk(KERN_ERR "%s copy_to_user error\n", LOG_PREFIX);

```

```

71
72         return -EFAULT;
73     }
74
75     memset(log , 0, LOG_SIZE);
76
77     return logLen;
78 }
79
80 static ssize_t analyzerWrite(struct file *file , const char __user *buf ,
81     size_t len , loff_t *offp)
82 {
83     printk(KERN_INFO "%s write called\n", LOG_PREFIX);
84     return 0;
85 }
86
87 static int analyzerRelease(struct inode *spInode , struct file *spFile)
88 {
89     printk(KERN_INFO "%s release called\n", LOG_PREFIX);
90     module_put(THIS_MODULE);
91     return 0;
92 }
93
94 static struct proc_ops fops = {
95     proc_read:    analyzerRead ,
96     proc_write:   analyzerWrite ,
97     proc_open:    analyzerOpen ,
98     proc_release: analyzerRelease
99 };

```

3.3. Реализация алгоритма предоставления информации о процессах пользователю

В листинге 3.2 представлена реализация алгоритма предоставления информации об изменении приоритетов, времени непрерывной работы, простоя и блокировки процессов пользователю.

Листинг 3.2 – Реализация алгоритма предоставления информации об изменении приоритетов, времени непрерывной работы, простоя и блокировки процессов пользователю

```

1  int main(int argc , char *argv [])
2  {
3      FILE *filePtr = NULL;
4      char log[LOG_SIZE] = {'\0'};
5

```

```

6      system("sudo insmod taskInfoGetter.ko");
7
8      for (int i = 0; i < LOGS_COUNT; i++)
9      {
10         filePtr = popen("cat /proc/processAnalyzer", "r");
11
12         if (filePtr == NULL)
13         {
14             printf("Error: can't execute cat for process analyzer");
15             return 1;
16         }
17
18         while (fgets(log, sizeof(log), filePtr) != NULL)
19             printf("%s", log);
20
21         pclose(filePtr);
22         sleep(DELAY_S);
23     }
24
25     system("sudo rmmod taskInfoGetter");
26
27     return EXIT_SUCCESS;
28 }

```

3.4. Makefile

В листинге 3.3 представлен Makefile для сборки компонентов программы.

Листинг 3.3 – Содержание Makefile

```

1  ifneq ($(KERNELRELEASE),)
2      obj-m := taskInfoGetter.o
3      taskInfoGetter-objs := md.o
4  else
5      CURRENT = $(shell uname -r)
6      KDIR = /lib/modules/$(CURRENT)/build
7      PWD = $(shell pwd)
8
9  default:
10     $(MAKE) -C $(KDIR) M=$(PWD) modules
11     make clean
12
13  logger:
14     gcc starterLogger.c -Wall -Werror -o sl.exe
15
16  clean:
17     @rm -f *.o *.cmd *.flags *.mod.c *.order *.mod *.symvers
18     @rm -f *.*.cmd *~ *.~*~ TODO.*

```

```

19      @rm -fR .tmp*
20      @rm -rf .tmp_versions
21
22  disclean: clean
23      @rm *.ko *.symvers *.mod
24  endif

```

3.5. Демонстрация работы

В листинге 3.4 представлена демонстрация работы программы загрузки модуля и получения информации.

Листинг 3.4 – Фрагмент вывода при запуске программы

```

1  ~~~~~1 TIME~~~~~
2  pid: 16      , comm:      migration/0
3  prio:  0, static_prio: 120, normal_prio (with scheduler policy):  0,
4  rt_priority:  99
5  run_delay: 1112861120
6  utime:      0 (ticks), stime:      8000000 (ticks)
7  sched_statistics:
8  wait_max:      8400, wait_count:      410, wait_sum:      3657080
9  iowait_count:      0, iowait_sum:      0
10 sleep_max:      3000, sum_sleep_runtime: 1101204040
11 block_max:      8400, sum_block_runtime: 1112861120, exec_max:      74000

```

В листинге 3.5 представлено содержание журнала ядра при загрузке и выгрузке модуля из системы.

Листинг 3.5 – Содержание журнала ядра

```

1  [ 465.991558] [PROCESS ANALYZER] module loaded
2  [ 465.996553] [PROCESS ANALYZER] open called
3  [ 465.996591] [PROCESS ANALYZER] read called
4  [ 465.996660] [PROCESS ANALYZER] read called
5  [ 465.996807] [PROCESS ANALYZER] release called
6  [ 476.001023] [PROCESS ANALYZER] open called
7  [ 476.001075] [PROCESS ANALYZER] read called
8  [ 476.001194] [PROCESS ANALYZER] release called
9  [ 486.005731] [PROCESS ANALYZER] open called
10 [ 486.005777] [PROCESS ANALYZER] read called
11 [ 486.005888] [PROCESS ANALYZER] release called
12 [ 496.010964] [PROCESS ANALYZER] open called
13 [ 496.011017] [PROCESS ANALYZER] read called
14 [ 496.011132] [PROCESS ANALYZER] release called
15 [ 506.016397] [PROCESS ANALYZER] open called
16 [ 506.016450] [PROCESS ANALYZER] read called
17 [ 506.016569] [PROCESS ANALYZER] release called

```

18	[	516.021766]	[PROCESS ANALYZER]	open called
19	[	516.021817]	[PROCESS ANALYZER]	read called
20	[	516.021934]	[PROCESS ANALYZER]	release called
21	[	526.051060]	[PROCESS ANALYZER]	module unloaded

4 Исследовательская часть

4.1. Описание проводимого исследования

Ниже приведены технические характеристики устройства, на котором выполнялось исследование.

- Операционная система: Debian GNU/Linux 12 (bookworm) x86_64, версия ядра 6.1.0-12-amd64.
- Объём оперативной памяти: 16 Гб.
- Процессор: Intel i5-9300H 2.4 ГГц.

Исследование проводилось на ноутбуке, включённом в сеть электропитания. Во время проведения исследования ноутбук был нагружен только встроенными приложениями окружения, а также исследуемыми программами.

Для исследования изменения приоритетов, времени непрерывной работы, простоя и блокировки процессов в Linux рассматривались три приложения различного характера: медиапроигрыватель VLC Media Player, веб-сервер и программа для выполнения математических вычислений. Для каждого приложения было произведено 5 итераций вывода информации о соответствующем процессе с интервалом 10 секунд.

4.2. Вывод информации о процессе, выполняющем воспроизведение видео с использованием VLC Media Player

Для исследования процесса воспроизведения видео с помощью медиапроигрывателя VLC Media Player был открыт видеофайл. Для того, чтобы определить идентификатор процесса и удостовериться в том, что данный файл используется только исследуемым приложением, использовалась утилита `lsuf`.

При анализе результатов было обнаружено, что среди процессов, отнесённых к задачам реального времени, отсутствует процесс, проигрывающий видео. По этой причине было принято решение убрать фильтрацию информации о процессах по их принадлежности к задачам реального времени.

В листинге 4.1 представлен фрагмент вывода разработанной программы, отражающий статистику выполнения процесса, проигрывающего видео с использованием VLC Media Player.

Листинг 4.1 – Информация о процессе, выполняющем воспроизведение видео с использованием VLC Media Player

```
1  ~~~~~1 TIME~~~~~
2  pid: 32820, comm:          totem
3  prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
4  rt_priority:    0
5  run_delay: 189644604, task_is_realtime: 0
6  utime: 30864000000 (ticks), stime:      9728000000 (ticks)
7  sched_statistics:
8  wait_max:      32000, wait_count:      150, wait_sum:      380000
9  iowait_count:   684000, iowait_sum: 189264604
10 sleep_max:      0, sum_sleep_runtime:      0
11 block_max:      32000, sum_block_runtime: 189644604, exec_max:      51800
12
13 ~~~~~2 TIME~~~~~
14 pid: 32820, comm:          totem
15 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
16 rt_priority:    0
17 run_delay: 190086883, task_is_realtime: 0
18 utime: 31216000000 (ticks), stime:      9812000000 (ticks)
19 sched_statistics:
20 wait_max:      32000, wait_count:      230, wait_sum:      582000
21 iowait_count:   1048800, iowait_sum: 189504883
22 sleep_max:      0, sum_sleep_runtime:      0
23 block_max:      32000, sum_block_runtime: 190086883, exec_max:      51800
24
25 ~~~~~3 TIME~~~~~
26 pid: 32820, comm:          totem
27 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
28 rt_priority:    0
29 run_delay: 190417525, task_is_realtime: 0
30 utime: 31580000000 (ticks), stime:      9904000000 (ticks)
31 sched_statistics:
32 wait_max:      32000, wait_count:      290, wait_sum:      734000
33 iowait_count:   1322400, iowait_sum: 189683525
34 sleep_max:      0, sum_sleep_runtime:      0
35 block_max:      32000, sum_block_runtime: 190417525, exec_max:      59800
36
37 ~~~~~4 TIME~~~~~
38 pid: 32820, comm:          totem
39 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
40 rt_priority:    0
41 run_delay: 191623392, task_is_realtime: 0
42 utime: 31904000000 (ticks), stime:      9988000000 (ticks)
43 sched_statistics:
44 wait_max:      32000, wait_count:      360, wait_sum:      912000
45 iowait_count:   1641600, iowait_sum: 190711392
46 sleep_max:      0, sum_sleep_runtime:      0
```

```

47 block_max:          32000, sum_block_runtime: 191623392, exec_max:      59800
48
49 ~~~~~~5 TIME~~~~~
50 pid: 32820, comm:          totem
51 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
52 rt_priority:    0
53 run_delay: 193040872, task_is_realtime: 0
54 utime: 32268000000 (ticks), stime:      10100000000 (ticks)
55 sched_statistics:
56 wait_max:      32000, wait_count:      440, wait_sum:      1114000
57 iowait_count:   2006400, iowait_sum: 191926872
58 sleep_max:      0, sum_sleep_runtime:      0
59 block_max:      32000, sum_block_runtime: 193040872, exec_max:      59800

```

По результатам исследования процесса, выполняющего воспроизведение видео с использованием VLC Media Player, было установлено, что в течение работы данного процесса его приоритеты не изменялись и не соответствовали задачам реального времени. Рассматриваемое приложение 99.4% времени блокировки (поле `sum_block_runtime`) затратило на ожидание завершения ввода-вывода (поле `iowait_sum`), оставшиеся 0.6% пришлось на ожидание в очереди на выделение ресурсов (поле `wait_sum`).

4.3. Вывод информации о процессе, выполняющем обработку сетевых запросов

Для исследования процесса обработки сетевых запросов был запущен специально разработанный веб-сервер. Далее на него была подана постоянная нагрузка, состоящая из запросов на передачу файлов по сети. Интенсивность нагрузки составляла 400 запросов в секунду. Для того, чтобы определить идентификатор исследуемого процесса, использовалась команда `ps`, для генерации запросов на веб-сервер — утилита `ab`.

В листинге 4.2 представлен фрагмент вывода разработанной программы, отражающий статистику выполнения процесса, обрабатывающего запросы на передачу файлов по сети.

Листинг 4.2 – Информация о процессе, выполняющем обработку сетевых запросов

```

1 ~~~~~~1 TIME~~~~~
2 pid: 51122, comm:          app
3 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
4 rt_priority:    0

```

```

5 run_delay: 383070792, task_is_realtime: 0
6 utime: 884000000 (ticks), stime: 3388000000 (ticks)
7 sched_statistics:
8 wait_max: 18800, wait_count: 557600, wait_sum: 636983200
9 iowait_count: 7714200, iowait_sum: 890145400
10 sleep_max: 150000, sum_sleep_runtime: 150000
11 block_max: 150000, sum_block_runtime: 1527278600, exec_max: 79380
12
13 ~~~~~~2 TIME~~~~~
14 pid: 51122, comm: app
15 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
16 rt_priority: 0
17 run_delay: 442082750, task_is_realtime: 0
18 utime: 1020000000 (ticks), stime: 3788000000 (ticks)
19 sched_statistics:
20 wait_max: 18800, wait_count: 646000, wait_sum: 738321400
21 iowait_count: 8941400, iowait_sum: 1031759440
22 sleep_max: 150000, sum_sleep_runtime: 150000
23 block_max: 150000, sum_block_runtime: 1770230840, exec_max: 79380
24
25 ~~~~~~3 TIME~~~~~
26 pid: 51122, comm: app
27 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
28 rt_priority: 0
29 run_delay: 498946337, task_is_realtime: 0
30 utime: 1120000000 (ticks), stime: 4164000000 (ticks)
31 sched_statistics:
32 wait_max: 22660, wait_count: 709600, wait_sum: 810705800
33 iowait_count: 9818000, iowait_sum: 1132912900
34 sleep_max: 150000, sum_sleep_runtime: 150000
35 block_max: 150000, sum_block_runtime: 1943768700, exec_max: 79380
36
37 ~~~~~~4 TIME~~~~~
38 pid: 51122, comm: app
39 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
40 rt_priority: 0
41 run_delay: 514424039, task_is_realtime: 0
42 utime: 1220000000 (ticks), stime: 4588000000 (ticks)
43 sched_statistics:
44 wait_max: 22660, wait_count: 773036, wait_sum: 883000000
45 iowait_count: 10694680, iowait_sum: 1234065200
46 sleep_max: 150000, sum_sleep_runtime: 150000
47 block_max: 150000, sum_block_runtime: 2117215200, exec_max: 85620
48
49 ~~~~~~5 TIME~~~~~
50 pid: 51122, comm: app
51 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
52 rt_priority: 0

```

```

53 run_delay: 573472791, task_is_realtime: 0
54 utime: 1292000000 (ticks), stime: 4980000000 (ticks)
55 sched_statistics:
56 wait_max: 22660, wait_count: 818000, wait_sum: 933780000
57 iowait_count: 11308000, iowait_sum: 1304800000
58 sleep_max: 150000, sum_sleep_runtime: 150000
59 block_max: 150000, sum_block_runtime: 2238730000, exec_max: 85620

```

По результатам исследования процесса, выполняющего обработку сетевых запросов, было установлено, что в течение работы данного процесса его приоритеты не изменялись и не соответствовали задачам реального времени. Рассматриваемое приложение 58.283% времени блокировки затратило на ожидание завершения ввода-вывода, 41.710% — на ожидание в очереди на выделение ресурсов, оставшиеся 0.007% пришлось на нахождение в состоянии прерываемого сна.

4.4. Вывод информации о процессе, выполняющем математические вычисления

Для исследования процесса выполнения математических вычислений была запущена программа, выполняющая решение дифференциальных уравнений. Для того, чтобы определить идентификатор исследуемого процесса, использовалась команда `ps`.

В листинге 4.3 представлен фрагмент вывода разработанной программы, отражающий статистику выполнения процесса, выполняющего математические вычисления.

Листинг 4.3 – Информация о процессе, выполняющем математические вычисления

```

1  ~~~~~1 TIME~~~~~
2  pid: 41386, comm: main
3  prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
4  rt_priority: 0
5  run_delay: 10159758, task_is_realtime: 0
6  utime: 9116000000 (ticks), stime: 36000000 (ticks)
7  sched_statistics:
8  wait_max: 7400, wait_count: 111800, wait_sum: 2129783400
9  iowait_count: 0, iowait_sum: 0
10 sleep_max: 0, sum_sleep_runtime: 0
11 block_max: 7400, sum_block_runtime: 2129783400, exec_max: 124400
12
13 ~~~~~2 TIME~~~~~

```

```

14 pid: 41386, comm:                main
15 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
16 rt_priority:    0
17 run_delay:    10302105, task_is_realtime: 0
18 utime: 9524000000 (ticks), stime:        40000000 (ticks)
19 sched_statistics:
20 wait_max:      7400, wait_count:          0, wait_sum: 2468612600
21 iowait_count:    0, iowait_sum:          0
22 sleep_max:      0, sum_sleep_runtime:      0
23 block_max:      7400, sum_block_runtime: 2468612600, exec_max:    124400
24
25 ~~~~~~3 TIME~~~~~
26 pid: 41386, comm:                main
27 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
28 rt_priority:    0
29 run_delay:    12794702, task_is_realtime: 0
30 utime: 10772000000 (ticks), stime:        40000000 (ticks)
31 sched_statistics:
32 wait_max:      7400, wait_count:          0, wait_sum: 2710633400
33 iowait_count:    0, iowait_sum:          0
34 sleep_max:      0, sum_sleep_runtime:      0
35 block_max:      7400, sum_block_runtime: 2710633400, exec_max:    124400
36
37 ~~~~~~4 TIME~~~~~
38 pid: 41386, comm:                main
39 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
40 rt_priority:    0
41 run_delay:    14617802, task_is_realtime: 0
42 utime: 13012000000 (ticks), stime:        48000000 (ticks)
43 sched_statistics:
44 wait_max:      7400, wait_count:          0, wait_sum: 2952654200
45 iowait_count:    0, iowait_sum:          0
46 sleep_max:      0, sum_sleep_runtime:      0
47 block_max:      7400, sum_block_runtime: 2952654200, exec_max:    124400
48
49 ~~~~~~5 TIME~~~~~
50 pid: 41386, comm:                main
51 prio: 120, static_prio: 120, normal_prio (with scheduler policy): 120,
52 rt_priority:    0
53 run_delay:    18906731, task_is_realtime: 0
54 utime: 14380000000 (ticks), stime:        52000000 (ticks)
55 sched_statistics:
56 wait_max:      7400, wait_count:          0, wait_sum: 3122068800
57 iowait_count:    0, iowait_sum:          0
58 sleep_max:      0, sum_sleep_runtime:      0
59 block_max:      7400, sum_block_runtime: 3122068800, exec_max:    124400

```

По результатам исследования процесса, выполняющего математические

вычисления, было установлено, что в течение работы данного процесса его приоритеты не изменялись и не соответствовали задачам реального времени. Рассматриваемое приложение не взаимодействовало с внешними устройствами и затрачивало всё время выполнения ($utime + stime$) на вычисления, блокируясь только по истечении кванта, предоставляемого системой, и помещении данного процесса в очередь на выполнение.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы был разработан загружаемый модуль ядра, позволяющий отслеживать изменения приоритетов, времени непрерывной работы, простоя и блокировки процессов в операционной системе Linux, а также были проанализированы приложения различного характера.

В ходе выполнения данной работы были решены следующие задачи.

- проанализированы и выбраны структуры ядра, содержащие необходимую информацию;
- проанализированы и выбраны методы передачи информации из модуля ядра в пространство пользователя;
- разработаны алгоритмы и структура программного обеспечения;
- исследованы приложения различного характера с помощью разработанного программного обеспечения.

Таким образом, все поставленные задачи были выполнены, поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Таненбаум Э., Бос Х.* Современные операционные системы. — 4-е изд. — Питер, 2015.
2. Real-Time group scheduling [Электронный ресурс]. — Режим доступа, URL: <https://docs.kernel.org/scheduler/sched-rt-group.html> (дата обращения: 3.12.2023).
3. Исходный код `/include/linux/sched.h` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/include/linux/sched.h> (дата обращения: 4.12.2023).
4. Исходный код `/include/linux/sched/prio.h` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/include/linux/sched/prio.h> (дата обращения: 4.12.2023).
5. Исходный код `/kernel/sched/core.c` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/kernel/sched/core.c> (дата обращения: 4.12.2023).
6. Исходный код `/kernel/sched/stats.h` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/kernel/sched/stats.h> (дата обращения: 18.12.2023).
7. Исходный код `/include/linux/sched/rt.h` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/include/linux/sched/rt.h> (дата обращения: 3.12.2023).
8. `sched(7)` — Linux manual page [Электронный ресурс]. — Режим доступа, URL: <https://man7.org/linux/man-pages/man7/sched.7.html> (дата обращения: 3.12.2023).
9. Исходный код `/include/linux/proc_fs.h` [Электронный ресурс]. — Режим доступа, URL: https://elixir.bootlin.com/linux/v6.1/source/include/linux/proc_fs.h (дата обращения: 3.12.2023).
10. Исходный код `/include/linux/uaccess.h` [Электронный ресурс]. — Режим доступа, URL: <https://elixir.bootlin.com/linux/v6.1/source/include/linux/uaccess.h> (дата обращения: 3.12.2023).

11. ISO/IEC 9899:1990 Programming languages — C [Электронный ресурс]. — Режим доступа, URL: <https://www.iso.org/standard/17782.html> (дата обращения: 16.12.2023).
12. Линус Торвальдс запланировал внедрение Rust в Linux 6.1 [Электронный ресурс]. — Режим доступа, URL: <https://www.linux.org.ru/news/kernel/16978439> (дата обращения: 16.12.2023).
13. GCC, the GNU Compiler Collection [Электронный ресурс]. — Режим доступа, URL: <https://gcc.gnu.org/> (дата обращения: 16.12.2023).
14. How to Compile a Linux Kernel [Электронный ресурс]. — Режим доступа, URL: <https://www.linux.com/topic/desktop/how-compile-linux-kernel-0> (дата обращения: 16.12.2023).

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД ПРОГРАММЫ

Листинг 4.4 – Исходный код загружаемого модуля ядра

```
1 #include <linux/delay.h>
2 #include <linux/init.h>
3 #include <linux/init_task.h>
4 #include <linux/kernel.h>
5 #include <linux/module.h>
6 #include <linux/proc_fs.h>
7 #include <linux/sched.h>
8 #include <linux/kthread.h>
9
10 #define PROC_FS_NAME "processAnalyzer"
11 #define LOG_PREFIX "[PROCESS ANALYZER]"
12 #define TMP_STRLEN 1024
13 #define LOGS_COUNT 5
14 #define DELAY_MS 10 * 1000
15 #define LOG_SIZE 524288
16
17 MODULE_LICENSE("GPL");
18 MODULE_AUTHOR("Inspirate789");
19
20 static struct proc_dir_entry *procFile;
21 static struct task_struct *kthread;
22 static char log[LOG_SIZE] = { 0 };
23
24 static int checkOverflow(char *fString, char *sString, int maxSize)
25 {
26     int sumLen = strlen(fString) + strlen(sString);
27
28     if (sumLen >= maxSize)
29     {
30         printk(KERN_ERR "%s not enough space in log (%d needed but %d\n", LOG_PREFIX, sumLen, maxSize);
31         available)\n", LOG_PREFIX, sumLen, maxSize);
32         return -ENOMEM;
33     }
34
35     return 0;
36 }
37
38 static int printTasks(void *arg)
39 {
40     struct task_struct *task;
41     size_t currentPrint = 1;
42     char currentString[TMP_STRLEN];
43
44     while (currentPrint <= LOGS_COUNT)
```

```

45     {
46         task = &init_task;
47         memset(currentString, 0, TMP_STRLEN);
48         snprintf(currentString, TMP_STRLEN, "~~~~~%lu
49             TIME~~~~~\n", currentPrint);
50         checkOverflow(currentString, log, LOG_SIZE);
51         strcat(log, currentString);
52
53         for_each_process(task)
54         {
55             if (task_is_realtime(task))
56             {
57                 memset(currentString, 0, TMP_STRLEN);
58                 snprintf(currentString, TMP_STRLEN,
59                     "pid: %-5d, comm: %15s\nprio: %3d, static_prio: %3d,
60                     normal_prio (with "
61                     "scheduler policy): %3d, rt_priority: %3d\n"
62                     "run_delay: %10lld\nutime: %10lld (ticks), stime:
63                     %15lld (ticks)\n"
64                     "sched_statistics:\nwait_max: %10llu, wait_count:
65                     %10llu, wait_sum: %10llu\n"
66                     "iowait_count: %10llu, iowait_sum: %10llu\n"
67                     "sleep_max: %10llu, sum_sleep_runtime: %10llu\n"
68                     "block_max: %10llu, sum_block_runtime: %10llu, exec_max:
69                     %10llu\n\n",
70                     task->pid, task->comm, task->prio, task->static_prio,
71                     task->normal_prio, task->rt_priority,
72                     task->sched_info.run_delay, task->utime, task->stime,
73                     task->stats.wait_max, task->stats.wait_count,
74                     task->stats.wait_sum,
75                     task->stats.iowait_count, task->stats.iowait_sum,
76                     task->stats.sleep_max, task->stats.sum_sleep_runtime,
77                     task->stats.block_max, task->stats.sum_block_runtime,
78                     task->stats.exec_max
79                     );
80                 checkOverflow(currentString, log, LOG_SIZE);
81                 strcat(log, currentString);
82             }
83         }
84
85         currentPrint++;
86         mdelay(DELAY_MS);
87     }
88
89     return 0;
90 }
91
92 static int analyzerOpen(struct inode *spInode, struct file *spFile)

```

```

93  {
94      printk(KERN_INFO "%s open called\n", LOG_PREFIX);
95      try_module_get(THIS_MODULE);
96      return 0;
97  }
98
99  static ssize_t analyzerRead(struct file *filep, char __user *buf,
100      size_t count, loff_t *offp)
101  {
102      ssize_t logLen = strlen(log);
103      printk(KERN_INFO "%s read called\n", LOG_PREFIX);
104
105      if (copy_to_user(buf, log, logLen))
106      {
107          printk(KERN_ERR "%s copy_to_user error\n", LOG_PREFIX);
108
109          return -EFAULT;
110      }
111
112      memset(log, 0, LOG_SIZE);
113
114      return logLen;
115  }
116
117  static ssize_t analyzerWrite(struct file *file, const char __user *buf,
118      size_t len, loff_t *offp)
119  {
120      printk(KERN_INFO "%s write called\n", LOG_PREFIX);
121      return 0;
122  }
123
124  static int analyzerRelease(struct inode *spInode, struct file *spFile)
125  {
126      printk(KERN_INFO "%s release called\n", LOG_PREFIX);
127      module_put(THIS_MODULE);
128      return 0;
129  }
130
131  static struct proc_ops fops = {
132      proc_read:    analyzerRead,
133      proc_write:   analyzerWrite,
134      proc_open:    analyzerOpen,
135      proc_release: analyzerRelease
136  };
137
138  static int __init md_init(void)
139  {
140      if (!(procFile = proc_create(PROC_FS_NAME, 0666, NULL, &fops)))

```

```

141     {
142         printk(KERN_ERR "%s proc_create error\n", LOG_PREFIX);
143         return -EFAULT;
144     }
145
146     kthread = kthread_run(printTasks, NULL, "taskPrintThread");
147     if (IS_ERR(kthread))
148     {
149         printk(KERN_ERR "%s kthread_run error\n", LOG_PREFIX);
150         return -EFAULT;
151     }
152
153     printk(KERN_INFO "%s module loaded\n", LOG_PREFIX);
154
155     return 0;
156 }
157
158 static void __exit md_exit(void)
159 {
160     remove_proc_entry(PROC_FS_NAME, NULL);
161     printk(KERN_INFO "%s module unloaded\n", LOG_PREFIX);
162 }
163
164 module_init(md_init);
165 module_exit(md_exit);

```

Листинг 4.5 – Исходный код программы для получения информации о процессах в пространстве пользователя

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/syscall.h>
5  #include <unistd.h>
6  #include <errno.h>
7
8  #define LOG_SIZE 8192
9  #define LOGS_COUNT 5
10 #define DELAY_S 10
11
12 int main(int argc, char *argv[])
13 {
14     FILE *filePtr = NULL;
15     char log[LOG_SIZE] = {'\0'};
16
17     system("sudo insmod taskInfoGetter.ko");
18
19     for (int i = 0; i < LOGS_COUNT; i++)

```

```

20     {
21         filePtr = popen("cat /proc/processAnalyzer", "r");
22
23         if (filePtr == NULL)
24         {
25             printf("Error: can't execute cat for process analyzer");
26             return 1;
27         }
28
29         while (fgets(log, sizeof(log), filePtr) != NULL)
30             printf("%s", log);
31
32         pclose(filePtr);
33         sleep(DELAY_S);
34     }
35
36     system("sudo rmmod taskInfoGetter");
37
38     return EXIT_SUCCESS;
39 }

```